

Valuación del 5.º tranche de un CDO sintético

Objetivo de la libreta

Esta libreta valúa el quinto tranche del CDO sintético del laboratorio, correspondiente al intervalo 12%–22%, manteniendo los supuestos del Excel base y usando $M = 100$ nodos de Gauss-Hermite como pide la tarea.

1. De un CDS a un CDO sintético

Un CDS transfiere riesgo de crédito: el comprador de protección paga una prima periódica y recibe compensación si ocurre un evento de crédito.

El laboratorio extiende esa misma lógica a un portafolio de CDS. Cuando el subyacente no son bonos físicos sino posiciones en CDS, hablamos de un CDO sintético. En este caso no se valúa un crédito individual, sino un portafolio homogéneo de 125 CDS cuyas pérdidas se reparten entre distintos tranches.

2. Datos de entrada declarados dentro de la libreta

Input	Fuente conceptual en el Excel	Uso en la valuación
125 CDS de 500,000	Portafolio CDS	Construir el valor del portafolio
Hazard rate	Datos	Probabilidad acumulada de default
Recovery rate / LGD	Datos	Pérdida por cada default
Correlación	Datos	Dependencia entre defaults
Tasa libre de riesgo	Datos	Descuento de flujos
Tranches	Datos	Attachment y detachment
$M = 100$	Requerimiento de la tarea	Integración numérica

Primero se declaran las variables base y después se construyen dentro del notebook las tres tablas que antes estaban separadas en archivos: portafolio, estructura de tranches y calendario de pagos.

```
In [1]: import math
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.stats import norm
from IPython.display import display

pd.set_option("display.float_format", lambda x: f"{x:,.10f}")

#supuestos y parámetros
notional_per_cds = 500_000
n_entities_input = 125
maturity_years = 1.0
premium_period = 0.25
hazard_rate = 0.00323
recovery_rate = 0.40
lgd = 1.0 - recovery_rate
correlation = 0.25
risk_free_rate = 0.03
M = 100

target_tranche = "12-22%"
attachment_target = 0.12
detachment_target = 0.22

# Valores de control que aparecen en el Excel original.
portfolio_value_from_excel = 62_500_000
n_entities_from_excel = 125

#tablas de datos
cds = pd.DataFrame({
    "cds_id": [f"CDS{i}" for i in range(1, n_entities_input + 1)],
    "notional": [notional_per_cds] * n_entities_input,
})

tranches = pd.DataFrame({
```

```
"tranche_name": ["0-3%", "3-6%", "6-9%", "9-12%", "12-22%", "22-100%"],
"attachment": [0.00, 0.03, 0.06, 0.09, 0.12, 0.22],
"detachment": [0.03, 0.06, 0.09, 0.12, 0.22, 1.00],
})

schedule = pd.DataFrame({
    "period": [1, 2, 3, 4],
    "t": [0.25, 0.50, 0.75, 1.00],
    "delta": [0.25, 0.25, 0.25, 0.25],
})

assumptions_table = pd.DataFrame([
    {"supuesto": "notional por CDS", "valor": notional_per_cds, "fuente conceptual": "Portafolio CDS"},
    {"supuesto": "número de entidades", "valor": n_entities_input, "fuente conceptual": "Datos"},
    {"supuesto": "vencimiento", "valor": maturity_years, "fuente conceptual": "Datos"},
    {"supuesto": "periodo de primas", "valor": premium_period, "fuente conceptual": "Datos"},
    {"supuesto": "hazard rate", "valor": hazard_rate, "fuente conceptual": "Datos"},
    {"supuesto": "recovery rate", "valor": recovery_rate, "fuente conceptual": "Datos"},
    {"supuesto": "LGD", "valor": lgd, "fuente conceptual": "Datos"},
    {"supuesto": "correlación", "valor": correlation, "fuente conceptual": "Datos"},
    {"supuesto": "tasa libre de riesgo", "valor": risk_free_rate, "fuente conceptual": "Datos"},
    {"supuesto": "M", "valor": M, "fuente conceptual": "tarea"},
])

display(assumptions_table)
display(cds.head())
display(tranches)
display(schedule)
```

	supuesto	valor	fuente conceptual
0	notional por CDS	500,000.0000000000	Portafolio CDS
1	número de entidades	125.0000000000	Datos
2	vencimiento	1.0000000000	Datos
3	periodo de primas	0.2500000000	Datos
4	hazard rate	0.0032300000	Datos
5	recovery rate	0.4000000000	Datos
6	LGD	0.6000000000	Datos
7	correlación	0.2500000000	Datos
8	tasa libre de riesgo	0.0300000000	Datos
9	M	100.0000000000	tarea

	cds_id	notional
0	CDS1	500000
1	CDS2	500000
2	CDS3	500000
3	CDS4	500000
4	CDS5	500000

	tranche_name	attachment	detachment
0	0-3%	0.0000000000	0.0300000000
1	3-6%	0.0300000000	0.0600000000
2	6-9%	0.0600000000	0.0900000000
3	9-12%	0.0900000000	0.1200000000
4	12-22%	0.1200000000	0.2200000000
5	22-100%	0.2200000000	1.0000000000

	period	t	delta
0	1	0.2500000000	0.2500000000
1	2	0.5000000000	0.2500000000
2	3	0.7500000000	0.2500000000
3	4	1.0000000000	0.2500000000

3. Reconstrucción del portafolio y controles de fidelidad

Aunque las tablas ya se construyen dentro del notebook, el valor total del portafolio no se captura como un número final aislado. Se recalcula a partir de los 125 CDS declarados arriba:

$$V = 125 \times 500,000 = 62,500,000$$

```
In [2]: portfolio_value = float(cds["notional"].sum())
n_entities = int(len(cds))

if abs(portfolio_value - portfolio_value_from_excel) > 1e-6:
    raise ValueError("La suma de los CDS no coincide con el valor del portafolio del Excel.")
if n_entities != n_entities_from_excel:
    raise ValueError("El número de CDS no coincide con el número de entidades del Excel.")

assumptions = {
    "portfolio_value": portfolio_value,
    "n_entities": n_entities,
    "maturity_years": maturity_years,
    "premium_period": premium_period,
    "hazard_rate": hazard_rate,
    "recovery_rate": recovery_rate,
    "lgd": lgd,
    "correlation": correlation,
    "risk_free_rate": risk_free_rate,
    "M": M,
}

portfolio_check = pd.DataFrame([
    {"valor_portafolio_recalculado": portfolio_value,
    "valor_portafolio_excel": portfolio_value_from_excel,
    "numero_entidades_recalculado": n_entities,
    "numero_entidades_excel": n_entities_from_excel,
    }])
portfolio_check
```

```
Out[2]:
```

	valor_portafolio_recalculado	valor_portafolio_excel	numero_entidades_recalculado	numero_entidades_excel
0	62,500,000.000000000	62500000	125	125

4. Tranche que se valúa

La tarea pide el quinto tranche, que corresponde al intervalo 12%-22%.

$$\text{Nocional del tranche} = (0.22 - 0.12) \times 62,500,000 = 6,250,000$$

Este tranche no absorbe las primeras pérdidas del portafolio. Sólo empieza a dañarse cuando las pérdidas acumuladas superan el 12% y queda completamente consumido al llegar al 22%.

```
In [3]: selected_tranche = tranches.loc[tranches["tranche_name"] == target_tranche].iloc[0]

attachment = float(selected_tranche["attachment"])
detachment = float(selected_tranche["detachment"])
tranche_width = detachment - attachment
tranche_notional = assumptions["portfolio_value"] * tranche_width
loss_per_default = assumptions["lgd"] / assumptions["n_entities"]

m_L = math.ceil(attachment * assumptions["n_entities"] / assumptions["lgd"])
m_H = math.ceil(detachment * assumptions["n_entities"] / assumptions["lgd"])

tranche_setup = pd.DataFrame([
    {"tranche": target_tranche,
    "attachment": attachment,
    "detachment": detachment,
    "ancho_tranche": tranche_width,
    "nocional_tranche": tranche_notional,
    "perdida_por_default": loss_per_default,
    "m_L": m_L,
    "m_H": m_H,
    }])
tranche_setup
```

```
Out[3]:
```

	tranche	attachment	detachment	ancho_tranche	nocional_tranche	perdida_por_default	m_L	m_H
0	12-22%	0.1200000000	0.2200000000	0.1000000000	6,250,000.000000000	0.0048000000	25	46

Lectura financiera de `m_L` y `m_H`

Cada default produce una pérdida aproximada de:

$$\frac{LGD}{n} = \frac{0.60}{125} = 0.0048 = 0.48\%$$

Por eso:

- alrededor de 25 defaults llevan la cartera al attachment del 12%;
- alrededor de 46 defaults llevan la cartera al detachment del 22%.

La implementación deja al tranche intacto antes de `m_L`, reduce su notional de forma lineal entre `m_L` y `m_H`, y lo deja agotado a partir de `m_H`.

```
In [4]: k_values = np.arange(max(0, m_L - 3), m_H + 4)
remaining_fraction = []
for k in k_values:
    if k < m_L:
        remaining_fraction.append(1.0)
    elif k < m_H:
        fraction = (detachment - k * loss_per_default) / tranche_width
        remaining_fraction.append(max(0.0, min(1.0, fraction)))
    else:
        remaining_fraction.append(0.0)

payoff_table = pd.DataFrame({
    "defaults_k": k_values,
    "perdida_portafolio": k_values * loss_per_default,
    "fraccion_viva_tranche": remaining_fraction,
})
payoff_table
```

Out[4]:

	defaults_k	perdida_portafolio	fraccion_viva_tranche
0	22	0.1056000000	1.0000000000
1	23	0.1104000000	1.0000000000
2	24	0.1152000000	1.0000000000
3	25	0.1200000000	1.0000000000
4	26	0.1248000000	0.9520000000
5	27	0.1296000000	0.9040000000
6	28	0.1344000000	0.8560000000
7	29	0.1392000000	0.8080000000
8	30	0.1440000000	0.7600000000
9	31	0.1488000000	0.7120000000
10	32	0.1536000000	0.6640000000
11	33	0.1584000000	0.6160000000
12	34	0.1632000000	0.5680000000
13	35	0.1680000000	0.5200000000
14	36	0.1728000000	0.4720000000
15	37	0.1776000000	0.4240000000
16	38	0.1824000000	0.3760000000
17	39	0.1872000000	0.3280000000
18	40	0.1920000000	0.2800000000
19	41	0.1968000000	0.2320000000
20	42	0.2016000000	0.1840000000
21	43	0.2064000000	0.1360000000
22	44	0.2112000000	0.0880000000
23	45	0.2160000000	0.0400000000
24	46	0.2208000000	0.0000000000
25	47	0.2256000000	0.0000000000
26	48	0.2304000000	0.0000000000
27	49	0.2352000000	0.0000000000

5. De hazard rate a probabilidad de default

El laboratorio usa un portafolio homogéneo: todos los CDS comparten el mismo hazard rate, recovery y correlación.

La probabilidad acumulada de default hasta una fecha t se calcula como:

$$Q(t) = 1 - e^{-ht}$$

donde h es el hazard rate anual del Excel. Esta probabilidad es todavía incondicional: resume el riesgo promedio de default antes de introducir escenarios sistémicos.

```
In [5]: def unconditional_default_probability(t: float, hazard_rate: float) -> float:
        return 1.0 - math.exp(-hazard_rate * t)

pd.DataFrame({
    "t": schedule["t"],
    "Q(t)": [unconditional_default_probability(t, assumptions["hazard_rate"]) for t in schedule["t"]],
})
```

Out[5]:

	t	Q(t)
0	0.2500000000	0.0008071741
1	0.5000000000	0.0016136966
2	0.7500000000	0.0024195681
3	1.0000000000	0.0032247892

6. Correlación y factor sistémico

En un CDO importa el riesgo de que varias entidades fallen juntas. Para capturar esa dependencia, el laboratorio usa un modelo gaussiano de un factor:

$$X_i = \sqrt{\rho}F + \sqrt{1 - \rho}\varepsilon_i$$

- F es el factor sistémico común;
- ε_i es el componente propio de cada entidad;
- ρ es la correlación común.

Para cada valor de F , la probabilidad condicional de default es:

$$Q(t | F) = N\left(\frac{N^{-1}(Q(t)) - \sqrt{\rho}F}{\sqrt{1 - \rho}}\right)$$

En escenarios sistémicos malos, la probabilidad condicional de default sube; en escenarios buenos, baja.

7. Integración con Gauss-Hermite y $M = 100$

Para promediar sobre los posibles valores del factor sistémico F , se usa cuadratura de Gauss-Hermite.

Los nodos representan escenarios posibles del factor y los pesos indican cuánto contribuye cada escenario al promedio. A diferencia de una simulación Monte Carlo, aquí no se generan escenarios aleatorios: se evalúan 100 puntos representativos de la normal estándar.

```
In [6]: def gauss_hermite_standard_normal(M: int) -> tuple[np.ndarray, np.ndarray]:
    x_nodes, raw_weights = np.polynomial.hermite.hermgauss(M)
    factor_nodes = np.sqrt(2.0) * x_nodes
    weights = raw_weights / np.sqrt(np.pi)
    return factor_nodes, weights

def conditional_default_probability(
    t: float,
    hazard_rate: float,
    correlation: float,
    factor_nodes: np.ndarray,
) -> np.ndarray:
    q_t = unconditional_default_probability(t, hazard_rate)
    threshold = norm.ppf(q_t)
    argument = (threshold - np.sqrt(correlation) * factor_nodes) / np.sqrt(1.0 - correlation)
    return norm.cdf(argument)

factor_nodes, weights = gauss_hermite_standard_normal(assumptions["M"])

pd.DataFrame({
    "factor_node": factor_nodes,
    "weight": weights,
}).head()
```

```
Out[6]:
```

	factor_node	weight
0	-18.9596362174	0.0000000000
1	-18.1355915269	0.0000000000
2	-17.4555874039	0.0000000000
3	-16.8504421965	0.0000000000
4	-16.2937419175	0.0000000000

8. Distribución del número de defaults

Condicionado al factor sistémico, el número total de defaults se modela como:

$$K | F \sim \text{Binomial}(n, q(F))$$

$$P(K=k | F) = \binom{n}{k} q(F)^k (1-q(F))^{n-k}$$

Con $n = 125$, el cálculo se realiza en logaritmos para mantener estabilidad numérica al trabajar con combinaciones binomiales grandes.

In [7]:

```
def binomial_probabilities(n: int, q: np.ndarray) -> np.ndarray:
    k = np.arange(n + 1)[:, None]
    q = q[None, :]

    log_comb = np.array([
        math.lgamma(n + 1) - math.lgamma(i + 1) - math.lgamma(n - i + 1)
        for i in range(n + 1)
    ][:, None])

    q_safe = np.clip(q, 1e-300, 1 - 1e-16)
    log_p = log_comb + k * np.log(q_safe) + (n - k) * np.log1p(-q_safe)
    return np.exp(log_p)

# Ejemplo de la distribución condicional en la primera fecha trimestral.
q_first_date = conditional_default_probability(
    t=float(schedule.iloc[0]["t"]),
    hazard_rate=assumptions["hazard_rate"],
    correlation=assumptions["correlation"],
    factor_nodes=factor_nodes,
)
probabilities_first_date = binomial_probabilities(assumptions["n_entities"], q_first_date)

pd.DataFrame({
    "k": np.arange(0, 8),
    "P(K=k | primer nodo F)": probabilities_first_date[:, 0],
})
```

Out[7]:

	k	P(K=k primer nodo F)
0	0	0.0000000000
1	1	0.0000000000
2	2	0.0000000000
3	3	0.0000000000
4	4	0.0000000000
5	5	0.0000000000
6	6	0.0000000000
7	7	0.0000000000

9. Notional remanente esperado del tranche

Para cada fecha y cada escenario sistémico, se calcula la fracción esperada del tranche que sigue viva.

Es importante no confundir esta variable:

$E(t) = \text{fracción esperada viva del tranche en } t$

Por tanto:

$1 - E(t) = \text{pérdida esperada del tranche en } t$

La función siguiente traduce la distribución de defaults en notional remanente esperado.

In [8]:

```
def expected_remaining_notional_fraction_given_factor(
    t: float,
    attachment: float,
    detachment: float,
    factor_nodes: np.ndarray,
) -> np.ndarray:
    q_conditional = conditional_default_probability(
        t=t,
        hazard_rate=assumptions["hazard_rate"],
        correlation=assumptions["correlation"],
        factor_nodes=factor_nodes,
    )
    probs = binomial_probabilities(assumptions["n_entities"], q_conditional)

    m_low = math.ceil(attachment * assumptions["n_entities"] / assumptions["lgd"])
    m_high = math.ceil(detachment * assumptions["n_entities"] / assumptions["lgd"])
    width = detachment - attachment

    expected_remaining = probs[:, m_low].sum(axis=0)

    for k in range(m_low, min(m_high, assumptions["n_entities"] + 1)):
        fraction = (detachment - k * assumptions["lgd"] / assumptions["n_entities"]) / width
        fraction = max(0.0, min(1.0, fraction))
```

```

        expected_remaining += probs[k] * fraction

    return expected_remaining

remaining_by_date = []
for t in schedule["t"]:
    expected_remaining_by_factor = expected_remaining_notional_fraction_given_factor(
        t=float(t),
        attachment=attachment,
        detachment=detachment,
        factor_nodes=factor_nodes,
    )
    remaining_unconditional = float(np.sum(weights * expected_remaining_by_factor))
    remaining_by_date.append({
        "t": t,
        "fraccion_viva_esperada": remaining_unconditional,
        "perdida_esperada": 1.0 - remaining_unconditional,
    })

pd.DataFrame(remaining_by_date)

```

Out[8]:

	t	fraccion_viva_esperada	perdida_esperada
0	0.2500000000	0.9999997807	0.0000002193
1	0.5000000000	0.9999983716	0.0000016284
2	0.7500000000	0.9999948374	0.0000051626
3	1.0000000000	0.9999884060	0.0000115940

10. Valuación de las tres patas

La valuación del tranche conserva la lógica de un CDS:

Pata de primas normales

$$A = \sum_j \Delta t_j E(t_j) e^{-rt_j}$$

El vendedor de protección cobra primas sobre el notional del tranche que sigue vivo.

Prima acumulada

$$B = \sum_j \frac{1}{2} \Delta t_j \left(E(t_{j-1}) - E(t_j) \right) e^{-r(t_j - \Delta t_j / 2)}$$

Si ocurre una pérdida entre fechas de pago, se aproxima que sucede a mitad del periodo y se reconoce prima acumulada.

Pata de protección

$$C = \sum_j \left(E(t_{j-1}) - E(t_j) \right) e^{-r(t_j - \Delta t_j / 2)}$$

Esta pata representa el valor presente esperado de las pérdidas del tranche.

```

In [9]: def value_tranche(
    attachment: float,
    detachment: float,
    schedule: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    previous_remaining = np.ones_like(factor_nodes)
    premium_leg_A = 0.0
    accrual_leg_B = 0.0
    protection_leg_C = 0.0
    rows = []

    for _, row in schedule.iterrows():
        t = float(row["t"])
        dt = float(row["delta"])

        current_remaining = expected_remaining_notional_fraction_given_factor(
            t=t,
            attachment=attachment,
            detachment=detachment,
            factor_nodes=factor_nodes,
        )

        discount_end = math.exp(-assumptions["risk_free_rate"] * t)
        discount_mid = math.exp(-assumptions["risk_free_rate"] * (t - dt / 2.0))

```



```

A_j = float(np.sum(weights * dt * current_remaining * discount_end))
B_j = float(np.sum(weights * 0.5 * dt * (previous_remaining - current_remaining) * discount_mid))
C_j = float(np.sum(weights * (previous_remaining - current_remaining) * discount_mid))

premium_leg_A += A_j
accrual_leg_B += B_j
protection_leg_C += C_j

remaining_unconditional = float(np.sum(weights * current_remaining))
rows.append({
    "periodo": int(row["period"]),
    "t": t,
    "delta": dt,
    "factor_descuento_fin": discount_end,
    "factor_descuento_medio": discount_mid,
    "fraccion_viva_esperada": remaining_unconditional,
    "perdida_esperada": 1.0 - remaining_unconditional,
    "incremento_A": A_j,
    "incremento_B": B_j,
    "incremento_C": C_j,
})

previous_remaining = current_remaining

fair_spread = protection_leg_C / (premium_leg_A + accrual_leg_B)
summary = pd.DataFrame([
    "tranche": target_tranche,
    "portfolio_value": assumptions["portfolio_value"],
    "n_entities": assumptions["n_entities"],
    "attachment": attachment,
    "detachment": detachment,
    "tranche_notional": tranche_notional,
    "M": assumptions["M"],
    "premium_leg_A": premium_leg_A,
    "accrual_leg_B": accrual_leg_B,
    "protection_leg_C": protection_leg_C,
    "fair_spread_decimal": fair_spread,
    "fair_spread_percent": fair_spread * 100.0,
    "fair_spread_bps": fair_spread * 10000.0,
])

return summary, pd.DataFrame(rows)

valuation_summary, period_detail = value_tranche(
    attachment=attachment,
    detachment=detachment,
    schedule=schedule,
)

display(period_detail)
display(valuation_summary.T)

```

	periodo	t	delta	factor_descuento_fin	factor_descuento_medio	fraccion_viva_esperada	perdi
0	1	0.2500000000	0.2500000000	0.9925280548	0.9962570225	0.9999997807	
1	2	0.5000000000	0.2500000000	0.9851119396	0.9888130446	0.9999983716	
2	3	0.7500000000	0.2500000000	0.9777512372	0.9814246877	0.9999948374	
3	4	1.0000000000	0.2500000000	0.9704455335	0.9740915363	0.9999884060	

	0
tranche	12-22%
portfolio_value	62,500,000.0000000000
n_entities	125
attachment	0.1200000000
detachment	0.2200000000
tranche_notional	6,250,000.0000000000
M	100
premium_leg_A	0.9814546611
accrual_leg_B	0.0000014181
protection_leg_C	0.0000113451
fair_spread_decimal	0.0000115595
fair_spread_percent	0.0011559452
fair_spread_bps	0.1155945218

11. Spread justo

El spread justo se obtiene imponiendo una valuación equilibrada:

$$s(A+B)=C$$

$$s=\frac{C}{A+B}$$

Es decir, el valor presente esperado de las primas cobradas debe compensar el valor presente esperado de las pérdidas del tranche.

```
In [10]: fair_spread_bps = float(valuation_summary.loc[0, "fair_spread_bps"])
print(f"Spread justo del tranche {target_tranche}: {fair_spread_bps:.6f} bps")
```

Spread justo del tranche 12-22%: 0.115595 bps